

I. Decomposer

System prompt

You decompose a natural-language drone task into a list of subtasks. Only output Python code.

INPUT FORMAT:

```
task = <short description>
skills = {
    <SkillName>: <ServiceTime>,
    ...
}
objects = {
    <ObjectName>: (<x>, <y>, <z>),
    ...
}
```

OUTPUT FORMAT:

```
subtasks = [
    {"name": <SubTaskName>,
     "skill": <SkillName>,
     "object": <ObjectName>,
     "service_time": <ServiceTime>},
    ...
]
```

User/Assistant prompts

3 decomposition examples

User prompt

task = "Inspect Tower and measure wind conditions at House2."

```
skills = {
    'CaptureRGBImage': 1.1,
    'CaptureThermalImage': 2.4,
    'InspectStructure': 1.3,
    'MeasureWind': 2.7
}
```

```
objects = {
    'House1': (4, 15, 39),
    'House2': (84, 37, 90),
    'SolarPanel': (72, 8, 84),
    'Tower': (53, 91, 57)
}
```

LLM generates subtasks

II. Allocator

System prompt

You allocate subtasks to capable drones. Only output Python code.

INPUT FORMAT:

```
drones = {
    <DroneName>: {
        "skills": [<SkillName>, ...],
        "pos": (<x>, <y>, <z>),
        "speed": <speed>
    },
    ...
}
subtasks = [{
    "name": <SubTaskName>,
    "skill": <SkillName>,
    "object": <ObjectName>,
    "service_time": <ServiceTime>
},
...
]
```

CONSTRAINTS:

- 1) A drone qualifies only if the skills listed for a drone include the skill required for the subtask.
- 2) Match by skills only, ignore position and speed.
- 3) If no drones qualify, set the 'drones' list to [] for that subtask.

OUTPUT FORMAT:

```
subtasks_with_drones = [{
    "name": <SubTaskName>,
    "skill": <SkillName>,
    "object": <ObjectName>,
    "service_time": <ServiceTime>,
    "drones": [<DroneName>, ...]
},
...
]
```

User/Assistant prompts

3 allocation examples

User prompt

```
drones = {
    "Drone1": {
        "skills": ["InspectStructure",
                   "CaptureImage"],
        "pos": (0, 0, 0),
        "speed": 1.0},
    "Drone2": {
        "skills": ["MeasureWind",
                   "CaptureImage"],
        "pos": (1, 0, 0),
        "speed": 1.0},
    "Drone3": {
        "skills": ["InspectStructure",
                   "MeasureWind"],
        "pos": (0, 1, 0),
        "speed": 1.0}}
subtasks = [
    {
        "name": "SubTask1",
        "skill": "InspectStructure",
        "object": "Tower",
        "service_time": 2.0},
    {
        "name": "SubTask2",
        "skill": "MeasureWind",
        "object": "House2",
        "service_time": 1.5}]
```

LLM generates subtasks_with_drones

III. Scheduler

System prompt

You are a task scheduler for a multi-drone system. Only output Python code.

INPUT FORMAT:

```
subtasks_with_drones = [
    {"name": <SubTaskName>, "skill": <SkillName>,
     "object": <ObjectName>, "service_time":
     <ServiceTime>, "drones": [<DroneName>, ...]},
    ...
]
travel_times = ...
```

CONSTRAINTS:

- 1) Each subtask appears exactly once in the schedule across all drones.
- 2) Time semantics:
 - a) departure_time is the departure time from the drone's current location; departure_time >= 0.
 - b) arrival_time is the arrival time at the task's object.
 - c) First task: arrival_time - departure_time == travel_times['drone_to_object'][Drone][Object].
 - d) For consecutive tasks from object A to B on the same drone: arrival_time - departure_time == travel_times['drone_object_to_object'][Drone][A][B].
 - e) finish_time is the time when the service is completed at the object: finish_time = arrival_time + service_time
- 3) For any single drone, services cannot overlap in time: current_finish_time ≤ next_departure_time
- 4) Objective: minimize makespan (max over all endTime).

OUTPUT FORMAT:

```
schedule = {
    <DroneName>: [
        {"name": <SubTaskName>,
         "object": <ObjectName>,
         "skill": <SkillName>,
         "departure_time": <DepartureTime>,
         "arrival_time": <ArrivalTime>,
         "finish_time": <FinishTime>},
        ...
    ],
    ...
}
```

User/Assistant prompts

3 scheduling examples

User prompt

```
subtasks_with_drones = [{
    "name": "SubTask1",
    "skill": "InspectStructure",
    "object": "Tower",
    "service_time": 2.0,
    "drones": ["Drone1", "Drone3"]},
    {"name": "SubTask2",
     "skill": "MeasureWind",
     "object": "House2",
     "service_time": 1.5,
     "drones": ["Drone2", "Drone3"]}]]
```

```
travel_times = {
    "drone_to_object": {
        "Drone1": {"Tower": 4.0, "House2": 6.0},
        "Drone2": {"Tower": 5.0, "House2": 3.0},
        "Drone3": {"Tower": 4.5, "House2": 4.0}},
    "drone_object_to_object": {
        "Drone1": {
            "Tower": {"House2": 2.5},
            "House2": {"Tower": 2.5}},
        "Drone2": {
            "Tower": {"House2": 2.0},
            "House2": {"Tower": 2.0}},
        "Drone3": {
            "Tower": {"House2": 1.5},
            "House2": {"Tower": 1.5}}}}
```

LLM generates schedule

Generated schedule

```
schedule = {
    "Drone1": [{
        "name": "SubTask1",
        "object": "Tower",
        "skill": "InspectStructure",
        "departure_time": 0.0,
        "arrival_time": 4.0,
        "finish_time": 6.0}],
    "Drone2": [{
        "name": "SubTask2",
        "object": "House2",
        "skill": "MeasureWind",
        "departure_time": 0.0,
        "arrival_time": 3.0,
        "finish_time": 4.5}],
    "Drone3": []}
```

User input

World configuration

Travel time calculator module